Delta, Spark, the Pandas API on Spark (see *Figure 1.3*), and MLflow are notable examples of consistent innovation, largely driven by their transparency as open source projects. We mention these specifically because they were all initially created by either the founders of Databricks or company members following its formation.

ML developers benefit significantly from this transparency, as it provides them with unparalleled flexibility, easy integration, and robust support from the open source community – all without the overhead of maintaining an open source full stack.

Starting development as a contractor using Databricks is super-fast compared to when companies require a fresh development environment to be set up. Some companies require a service request to install Python libraries. This can be a productivity killer for data scientists. In Databricks, many of your favorite libraries are pre-installed and ready to use, and of course, you can easily install your own libraries as well.

Additionally, there is a large and vibrant community of Databricks users. The Databricks community website is an excellent resource to ask and answer questions about anything related to Databricks. We've included a link in the *Further reading* section at the end of this chapter.

```
import pandas as pd

df = pd.read_csv(r'/folder/single.csv', sep=',', decimal='.')
df.columns = ['a', 'b', 'c']
df['a2'] = df.a * df.a
spark.createDataFrame(df).write.saveAsTable("pandas_as_table")
```

```
import pyspark.pandas as ps

df = ps.read_csv(r'/folder-of-csv-files/', sep=',', decimal='.')
df.columns = ['a','b','c']
df['a2'] = df.a * df.a
df.to_table("pyspark_pandas_as_table")
```

Figure 1.3 – The pandas API on Spark

The pandas API on Spark is nearly identical syntax to standard pandas, making distributed computing with Spark easier to learn for those who have written pandas code in Python

While continuing with a focus on transparency, let's move on to Databricks **AutoML**.